UNIT 6

Q1) Explain in detail Decidable problems concerning regular languages, and Decidable problems concerning context free languages.

6.1 Decidability

SPPU : May-06,10,15,16,19, Dec.-07,10,11,12,13, Marks 12

- If a language is recursive then it is called **decidable language** and if the language is not recursive then such a language is called **undecidable language.**
- The class of decidable problems are also called as **solvable problems** and the class of undecidable problems is called **unsolvable problems.**

6.1.1 Decidability Problems Concerning Regular Languages

SPPU : May-15, 16

The decidability problems concerning regular languages can be understood with the help of following theorems.

Theorem 1 : Prove that following decision problems are recursive
(i) Two DFA's are equivalent or Not (ii)NFA Accepts a word or not

SPPU : May-15 End Sem, Marks 10, May-16, End Sem, Marks 5

Proof : (i)

- To prove that the given decision problem is recursive. We require a TM T(M) which simulates the DFA.
- Let, M_1 and M_2 are two DFAs. Consider the Turing machines $T(M_1)$ and $T(M_2)$ Obtain set of strings accepted by $T(M_1)$ and rejected by $T(M_2)$. That is
  $$S_1 = T(M_1) \cap \overline{T(M_2)}.$$
- Similarly obtain the set of strings which $T(M_2)$ accepts and $T(M_1)$ rejects. That is
  $$S_2 = T(M_2) \cap \overline{T(M_1)}.$$
- If these two sets are empty then it shows that $L(M_1)$ and $L(M_2)$ are equivalent and can be simulated by TM for the set of languages as.
  $$L(M) = L(M_1) \cap \overline{L(M_2)} \cup \overline{L(M_1)} \cap L(M_2). \text{ The } L(M) = \phi.$$
- This shows that two DFA's are equivalent or not is a recursive problem.

(ii) Let, M be the NFA and input word w which decides if M accepts w. Convert this NFA to DFA. Then run the algorithm for DFA on Turing Machine T(M). The TM T(M) accepts w if and only if M reaches a final state on w. This shows that NFA accepts a word or not is recursive.

Theorem 2 : Prove that following decision problems is recursive : DFA Accepts a word or not.

SPPU : May-15, End Sem, Marks 5

Proof :

Let, M be the DFA and input word w which decides if M accepts w. Simulate an algorithm for DFA to run on Turing machine T(M). The TM T(M) accepts w if and only if M reaches a final state for w otherwise rejects. This show that DFA accepts a word or not is recursive.

6.1.2 Decidability Problems Concerning Context Free Languages

SPPU : May-06,10,15,16, Dec.-07,11,12,13

The undecidable problems about CFG are -

1) Whether or not two different context free languages define the same language ?
2) Whether given CFL is ambiguous or not ?
3) Whether complement of given CFL is context free language ?
4) Whether the intersection of two context free languages is context free ?

There is no algorithm to answer these questions. Hence these problems are known as **undecidable** problems.

Theorem 3 : Show that the following problem is undecidable. "Given two CFGs G_1 and G_2, is $L(G_1) \cap L(G_2) = \phi$".

Proof :

Let $L(G_1)$ and $L(G_2)$ be two CFLs. The CFL is not closed under intersection. That means $L(G_1) \cap L(G_2)$ does not necessarily generates CFL. This also implies that there is no algorithm which can tell whether $L(G_1) \cap L(G_2) = \phi$. Hence it is said that $(L(G_1) \cap L(G_2) = \phi$ is undecidable.

Theorem 4 : Prove that CFG G generates the string w or not

SPPU : May-15, End Sem, Marks 5, May-16, End Sem, Marks 5

Proof :

Convert CFG G to G″ in chomsky normal form. Now the string $w \in L(G'')$ iff w can be derived in $2|w|-1$ steps where none of the intermediate string is of length more than $|w|$. If these derived step will derive w, the T(M) accepting L(G″) will accept otherwise rejects. This shows CFG G generates string w or not is recursive.

Q2) **Measuring Complexity**

6.3 Measuring Complexity

SPPU : Dec-17,18, Marks 6

There are two groups in which a problem can be classified. The first group consists of the problems that can be solved in **polynomial time**. For example : searching of an element from the list O(logn), sorting of elements O(logn).

The second group consists of problems that can be solved in **non-deterministic polynomial time**. For example : Knapsack problem $O(2^{n/2})$ and Travelling Salesperson problem $(O(n^2 2^n))$.

- Any problem for which answer is either yes or no is called decision problem. The algorithm for decision problem is called **decision algorithm**.
- Any problem that involves the identification of optimal cost (minimum or maximum) is called optimization problem. The algorithm for optimization problem is called **optimization algorithm**.

- **Definition of P** - Problems that can be solved in polynomial time. ("P" stands for polynomial). The polynomial time is nothing but the time expressed in terms of polnomial.
  Examples - Searching of key element, Sorting of elements, All pair shortest path.
- **Definition of NP** - It stands for "non-deterministic polynomial time". Note that NP does not stand for "non-polynomial".
- Examples - Travelling Salesperson problem, Graph coloring problem, Knapsack problem, Hamiltonian circuit problems.
- The NP class problems can be further categorized into NP-complete and NP hard problems.

- A problem D is called **NP-complete** if -
  i) It belongs to class NP
  ii) Every problem in NP can also be solved in polynomial time.
- If an NP-hard problem can be solved in polynomial time then all NP-complete problems can also be solved in polynomial time.

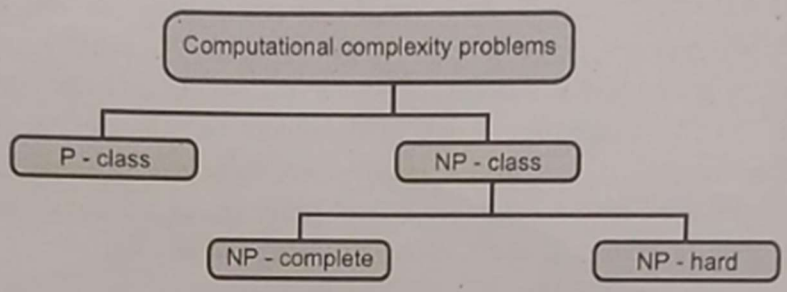

Fig. 6.3.1 Complexity classes

- All NP-complete problems are NP-hard but all NP-hard problems cannot be NP-complete.
- The NP class problems are the decision problems that can be solved by non-deterministic polynomial algorithms.
- **Computational complexity** - The computational problems is an infinite collection of instances with a solution for every instance.

In computational complexity the solution to the problem can be "yes" or "no" type. Such type of problems are called **decision problem.** The computational problems can be **function problem**. The function problem is a computational problem where single output is expected for every input. The output of such type of problems is more complex than the decision problem.

The computational problems also consists of a class of problems, whose output can be obtained in polynomial time.

- **Complexity classes -** The complexity classes is a set of problems of related complexity. It includes function problems, P classes, NP classes, optimization problem.
- **Intractability -** Problems that can be solved by taking a long time for their solutions is known as intractable problems.

If NP is not same as P then NP complete problems are called intractable problems.

Difference between P class and NP class problems

P Class	NP Class Problem
Problems that can be solved in polynomial time are called P-class problems.	Problems that can be solved in non-deterministically Polynomial time are called NP class problem.
These are simple to solve.	These are complex to solve.
Examples : Searching an element from unordered list, sorting the elements.	**Examples :** Traveling salesperson problem, knapsack problem.

Q3) **Cooks Theorem**

6.12 Cook's Theorem

Scientist Stephen Cook in 1971 stated that **Boolean Satisfiability (SAT) problem is NP complete.**

Proof :

Any instance of Boolean satisfiability problem is a boolean expression in which Boolean variables are combined using Boolean operators. An expression is satisfiable if its value results to be true on some assignments of Boolean variables.

The Boolean satisfiability problem is in NP. This is because a non-deterministic algorithm can guess an assignment of truth values of variables. This algorithm can also determine the value of expression for corresponding assignment and can accept if entire expression is true.

The algorithm is composed of -

- Input tape wherein tape is divided in finite number of cells.
- The read/write head which reads each symbol from tape.
- Each cell contain only one symbol at a time.
- Computation is performed in number of states.
- The algorithm terminates when it reaches to accept state.

The conjunction clauses for Boolean expression are given in following table.

Clauses	Meaning	Time
T_{ij0}	Cell i of input tape contains symbol j.	O(P(n))
Q_{s0}	Initial state.	O(1)
H_{00}	Initial position of tape head.	O(1)
$T_{ijk} = T_{ij(k+1)} \vee H_{ik}$	Tape remains unchanged unless written.	$O(P(n^2))$
$Q_{qk} \rightarrow \neg Q_{qk}$	One state at a time.	O(P(n))
$H_{ik} \rightarrow \neg H_{ik}$	One read / write head position at a time.	$O(P(n^2))$
$T_{ijk} \rightarrow \neg T_{ijk}$	One symbol per tape cell at a time.	$O(P(n^2))$

Note that H denotes head, Q denotes states and T denotes tape. The disjunction clause for this algorithm can be written as :

Clause	Meaning	Time
$(H_{ik} \wedge Q_{qk} \wedge T_{ijk}) \rightarrow (H_{(i+j)(k+1)} \wedge Q_{q(K+1)} \wedge T_{ij(K+1)})$	Possible transitions	$O(P(n^2))$
Disjunction of all clauses Q_f	Moving to accept state	O(1)

If boolean expression is satisfiable, then there is an accepting state in the algorithm.

Thus the proof shows that boolean satisfiability problem can be solved in polynomial time. Hence all problems in NP could be solved in polynomial time. And hence the complexity class NP could be equal to P.

Q4)**Node Cover Problem**

6.13 Node-Cover Problem

The node cover problem is to find node cover of minimum size in a given graph. The word node cover means each node covers its incident edges. Thus by node cover we expect to choose the set of vertices which cover all the edges in a graph.

For example : Consider a graph G as given below.

Now we will select some arbitrary edge and delete all the incident edges. Repeat this process until all the incident edges get deleted.

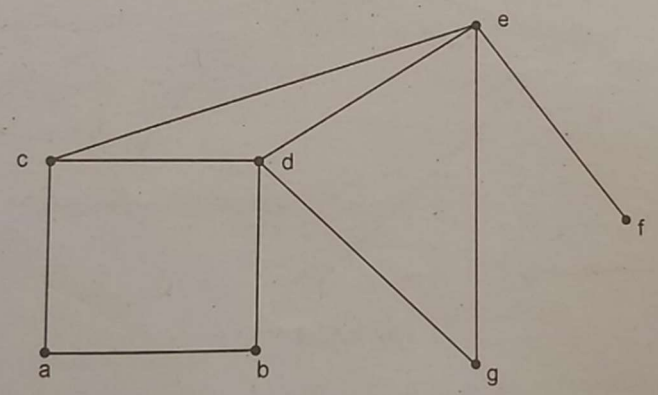

Fig. 6.13.1

- **Step 1 :** Select an edge a-b and delete all the incident edges to node a or b.

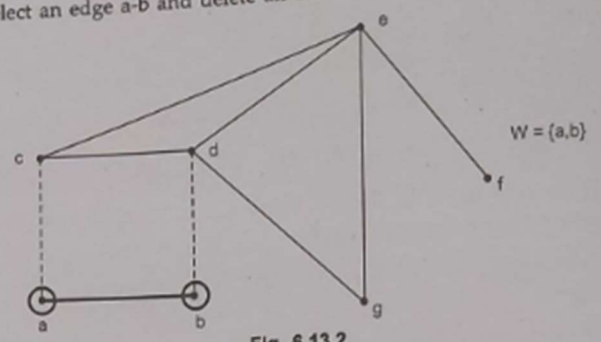

Fig. 6.13.2

- **Step 2 :** Select an edge c-e and delete all the incident edges to node c or e.

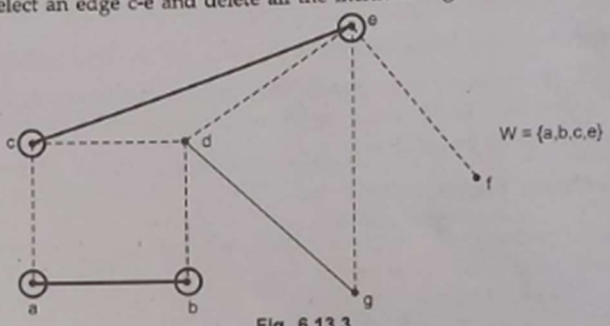

Fig. 6.13.3

- **Step 3 :** Select an edge d-g. All the incident edges are already deleted.

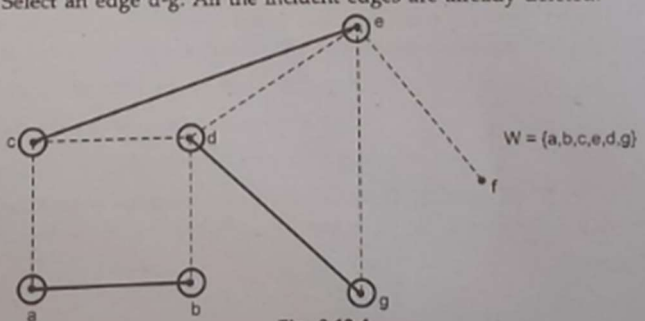

Fig. 6.13.4

Thus we obtain node cover as {a, b, c, d, e, g}.

Q5) Example of Undecidable problem

Theorem 5 : Let G be a CFG and r be a regular expression. Show that the problem
1) L (G) = L (r) 2) L (r) ∈ L (G) are undecidable

SPPU : May-06, 10, Dec.-07, Marks 12, Dec.-11, Marks 8

Proof :

1) Let, L (G) is a context free language and L(r) represents a regular language having regular expression. The post's correspondence problem PCP has some instance which becomes a solution for given language L(r) can be accepted by the CFG for L(G). It is also possible that there is no solution instance of PCP. Such an instance is not accepted by CFG. Thus there may exist a CFG for L (r) or there may not be a CFG for L(r). Hence we can state that L(G) = L(r) is undecidable.

2) Let L(r) represents a regular language for some regular expression r. The L(G) denotes the context free language for a CFG G. There may be some strings of language L(r) that are acceptable by G. Then we can state L(r) ∈ L(G).

But since these are arbitrary strings, there may be some strings of L((r) which are not acceptable by L(G). Hence L(r) ∉ L(G).

Thus it is not possible to determine whether L(r) ∈ L(G). Hence it is undecidable

Q6) Traveling Salesman Problem (TSP)

The **Traveling Salesman Problem (TSP)** is one of the most well-known and studied problems in computer science and optimization theory. It falls under the category of **combinatorial optimization problems**, where the goal is to find an optimal solution from a finite set of possible solutions.

Problem Definition: Given a set of **cities** (or nodes) and the **distances** between each pair of cities, the objective is to find the **shortest possible route** that visits every city exactly once and returns to the starting city.

Example:

Imagine a traveling salesman with 4 cities: A, B, C, and D, and the distances between them as follows:

- $d(A, B) = 10$
- $d(A, C) = 15$
- $d(A, D) = 20$
- $d(B, C) = 35$
- $d(B, D) = 25$
- $d(C, D) = 30$

The goal is to find the shortest route that starts at one city, visits every other city exactly once, and then returns to the starting city. For example, one possible route could be: $A \to B \to D \to C \to A$, and the objective is to calculate the total distance for all possible routes and select the one with the least distance.

Why is TSP an NP-Class Problem?

1. NP-Class (Nondeterministic Polynomial Time): A problem is in NP if, given a proposed solution, we can verify whether it is correct or not in polynomial time. This means that for any given solution to the problem, we can check whether the solution is valid or not in a time that is a polynomial function of the size of the input.
2. NP-Hard: A problem is NP-hard if solving it is at least as hard as solving any problem in NP. This means that an NP-hard problem might or might not belong to NP, but it is at least as difficult as any problem in NP.
3. NP-Complete: A problem is NP-complete if it is both in NP and NP-hard. These are the "hardest" problems in NP.

Justification for TSP being an NP-Class Problem:

1. TSP is in NP: Given a specific route (i.e., an ordering of cities), we can easily calculate the total distance of the route and verify if it satisfies the condition of being the shortest route in polynomial time (by simply adding up the distances between consecutive cities). Therefore, checking if a given solution (a route) is valid can be done in polynomial time.
2. TSP is NP-Hard: The Traveling Salesman Problem is NP-hard because it is as hard as any problem in NP. It can be shown that solving TSP is at least as difficult as any other NP problem. In fact, many other NP problems, such as the Hamiltonian Cycle Problem, are reducible to TSP. That means if we could solve TSP efficiently (in polynomial time), we could also solve other NP problems efficiently. Hence, TSP is NP-hard.
3. TSP is NP-Complete: The decision version of the TSP (i.e., whether there exists a route whose total distance is less than or equal to a given value) is NP-complete. This is because:
    - It is in NP (we can verify a given solution in polynomial time).
    - It is NP-hard (we can reduce other NP problems to TSP).

Q7) Satisfiability Problem (SAT)

The Satisfiability Problem (SAT) is one of the most fundamental problems in computer science and logic. It is a problem in propositional logic and plays a crucial role in computational complexity theory. The SAT problem asks whether a given Boolean formula can be satisfied by some assignment of truth values to its variables.

Problem Definition: Given a Boolean formula (in the form of a conjunctive normal form (CNF) expression), the task is to determine whether there exists a truth assignment to the variables such that the entire formula evaluates to true.

A Boolean formula is a logical expression built using AND (∧), OR (∨), and NOT (¬) operators applied to Boolean variables. These variables can have only two possible values: True (T) or False (F).

- A clause is a disjunction (OR) of literals.
- A CNF is a conjunction (AND) of clauses, where each clause is a disjunction of literals.

The SAT problem is said to be satisfiable if there exists some assignment of truth values to the variables that makes the formula true. If no such assignment exists, the formula is unsatisfiable.

Formal Definition:

Formal Definition:

Given a formula in CNF:

$$F = (L_1 \vee L_2 \vee \cdots \vee L_k) \wedge (L_{k+1} \vee L_{k+2} \vee \cdots \vee L_m) \wedge \cdots \wedge (L_p \vee L_q \vee \ldots)$$

where L1,L2,... are literals

Consider the Boolean formula:

$$F = (A \vee B \vee \neg C) \wedge (\neg A \vee C) \wedge (B \vee C)$$

Here, we have the following clauses:

- Clause 1: $(A \vee B \vee \neg C)$
- Clause 2: $(\neg A \vee C)$
- Clause 3: $(B \vee C)$

We need to determine if there exists an assignment of truth values to A, B, and C that makes the entire formula true.

Example

Step-by-Step Solution:

1. **Clause 1:** $(A \vee B \vee \neg C)$
   - This clause is **true** if at least one of A, B, or $\neg C$ is true.
2. **Clause 2:** $(\neg A \vee C)$
   - This clause is **true** if at least one of $\neg A$ or C is true.
3. **Clause 3:** $(B \vee C)$
   - This clause is **true** if at least one of B or C is true.

Checking Satisfiability:

We now check for a truth assignment that satisfies all three clauses simultaneously:

- Try assigning $A = \text{True}$, $B = \text{True}$, and $C = \text{False}$:
  - Clause 1: $A \vee B \vee \neg C$ becomes $\text{True} \vee \text{True} \vee \text{True} = \text{True}$ (satisfied).
  - Clause 2: $\neg A \vee C$ becomes $\text{False} \vee \text{False} = \text{False}$ (not satisfied).
  - Clause 3: $B \vee C$ becomes $\text{True} \vee \text{False} = \text{True}$ (satisfied).

  This assignment does not satisfy Clause 2, so it is not a valid solution.
- Now, try assigning $A = \text{False}$, $B = \text{True}$, and $C = \text{True}$:
  - Clause 1: $A \vee B \vee \neg C$ becomes $\text{False} \vee \text{True} \vee \text{False} = \text{True}$ (satisfied).
  - Clause 2: $\neg A \vee C$ becomes $\text{True} \vee \text{True} = \text{True}$ (satisfied).
  - Clause 3: $B \vee C$ becomes $\text{True} \vee \text{True} = \text{True}$ (satisfied).

  This assignment satisfies all three clauses, so **the formula is satisfiable**.

Justification:

The formula $F = (A \vee B \vee \neg C) \wedge (\neg A \vee C) \wedge (B \vee C)$ is **satisfiable** because we found an assignment of truth values to A, B, and C that makes the formula true.

NP-Class of SAT:

The Satisfiability Problem (SAT) is NP-complete, meaning:

1. SAT is in NP: Given a truth assignment for the variables, we can verify if the formula evaluates to true in polynomial time.
2. SAT is NP-hard: Any problem in NP can be reduced to SAT in polynomial time. In other words, if we can solve SAT efficiently, we can solve any NP problem efficiently.

Q8) What is Polynomial time reduction? Explain it with a suitable example.[6]

...reducibility with example. SPPU : May-18, End Sem, Marks 8

6.8 Polynomial Time Reduction

- To prove whether particular problem is NP complete or not we use polynomial time reducibility. That means if

$A \xrightarrow{Poly} B$ and $B \xrightarrow{Poly} C$ then $A \xrightarrow{Poly} C$.

- The **reduction** is an important task in NP completeness proofs. This can be illustrated by Fig. 6.8.1.
- Various types of reductions are

Local replacement - In this reduction A→B by dividing input to A in the form of components and then these components can be converted to components of B.

Component design - In this reduction A→B by building special component for input B that enforce properties required by A.

The reduction can be denoted as $A \leq_T^P B$.

Importance:

- **Polynomial-time reduction** is often used to prove that certain problems are **NP-hard**. If a known NP-hard problem can be reduced to a new problem in polynomial time, the new problem is also NP-hard.
- It allows us to transfer the difficulty of solving one problem to another.

Steps Involved in Polynomial Time Reduction:

1. **Transformation of Instance**: Convert the input of problem A into an input for problem B in polynomial time.
2. **Solving B**: Solve the transformed instance of B.
3. **Using the Solution**: Use the solution to problem B to derive a solution to the original problem A.

Q9) Reducibility and MAPPING reducibility

SPPU : May-16, End Sem, Marks 8

6.6 Reducibility

Language A is reducible to language B (represented as $A \leq B$) if there exists a function f which will convert strings in A to strings in B as :

$$W \in A \iff f(W) \in B$$

1: If $A \leq B$ is decidable then A is also decidable.

2: If $A \leq B$ and A is undecidable then B is also undecidable.

6.7 Mapping Reducibility

SPPU : May-18, Marks 8

- Reduction is a technique in which if a problem P1 is reduced to a problem P2 then any solution of P2 solves P1.
- In general, if we have an algorithm to convert and instance of a problem P1 to instance of a problem P2 that have the same answer then it is called as P1 reduces P2.
- Hence if P1 is not recursive then P2 is also not recursive. Similarly if P1 is not recursively enumerable then P2 also is not recursively enumerable.

Theorem 1 : If P1 is reduced to P2 then
i) If P1 is undecidable then P2 is also undecidable. ii) If P1 is non-RE then P2 is also non-RE.

Proof :

i) Consider an instance w of P1. Then construct an algorithm such that the algorithm takes instance w as input and coverts it into another instance x of P2. Then apply that algorithm to check whether x is in P2, if algorithm answers "yes" then that means x is in P2, similarly we can also say that w is in P1. Since we have obtained P2 after reduction of P1. Similarly if algorithm answers "no" then x is not in P2, that also means w is not in P1. This proves that if P1 is undecidable then P1 is also undecidable.

ii) We assume that P1 is non RE but P2 is RE. Now construct an algorithm to reduce P1 to P2, but by this algorithm P2 will be recognized. That means there will be a turing machine that says "yes" if the input is P2 but may or may not halt for the input which is not in P2. As we know that one can convert instance of w in P1 to an instance x in P2. Then apply a TM to check whether x is in P2. If x is accepted that also means w is accepted. This procedure describes a TM whose language is P1 if w is in P1 then x is also in P2 and if w is not in P1 then x is also not in P2. This proves that if P1 is non-RE then P2 is also non-RE.

q10) What do you mean by NP-Complete problems? List the problems in this class and explain any one problem in detail.

NP-Complete problems are a class of decision problems that are both:

1. In NP: They can be solved in nondeterministic polynomial time. This means that there exists a non-deterministic Turing machine that can solve the problem in polynomial time.
2. NP-hard: Every problem in NP can be reduced to this problem in polynomial time. In simpler terms, any problem in NP can be transformed into an NP-complete problem, meaning that solving an NP-complete problem efficiently (in polynomial time) would allow us to solve all NP problems efficiently.

Key Points about NP-Complete Problems:

- NP-complete problems are the hardest problems in NP, meaning that if we can find a polynomial-time solution for any NP-complete problem, we would be able to solve all NP problems in polynomial time.
- These problems are significant in computational theory because there is no known efficient (polynomial time) algorithm to solve them, and it is believed that no such algorithm exists (although this is unproven).

List of NP-Complete Problems:

1. Boolean Satisfiability Problem (SAT)
2. 3-SAT Problem
3. Travelling Salesman Problem (TSP)
3. Knapsack Problem
5. Hamiltonian Cycle Problem

Hamiltonian Cycle Problem: The **Hamiltonian Cycle** problem is one of the most well-known NP-complete problems in graph theory. It involves finding a cycle in a graph that visits each vertex exactly once and returns to the starting vertex. In other words, the Hamiltonian Cycle is a closed loop that passes through every vertex of the graph exactly once.

Problem Definition: Given an undirected graph G=(V,E) where:

- V is the set of vertices, and
- E is the set of edges.

The task is to determine if there exists a cycle in the graph that visits each vertex exactly once and returns to the starting vertex. This cycle is called a **Hamiltonian cycle**.

Input: An undirected graph G=(V,E).
Output:
YES if there exists a Hamiltonian cycle in the graph.
NO if no such cycle exists.
Why is the Hamiltonian Cycle Problem NP-Complete?

The Hamiltonian Cycle Problem is NP-complete because:
1. Hamiltonian Cycle is in NP: A given cycle can be verified in polynomial time. If a candidate cycle is given, we can check if it visits every vertex exactly once and forms a cycle in polynomial time by verifying each edge in the cycle.

2. Hamiltonian Cycle is NP-Hard: The problem is NP-hard because any problem in NP can be reduced to the Hamiltonian Cycle problem in polynomial time. This was proven through the reduction of the Travelling Salesman Problem (TSP) to the Hamiltonian Cycle problem. In the TSP, we are tasked with finding a minimum-cost Hamiltonian cycle, which is a more complex version of the Hamiltonian Cycle problem. Solving the Hamiltonian Cycle problem efficiently would help solve the TSP and other NP problems in polynomial time.

Thus, the Hamiltonian Cycle Problem is NP-complete because:

- It is in NP.
- It is NP-hard (any NP problem can be reduced to it in polynomial time).